C/C++-Kurs für Kinder ab der 5. Klasse

Jürgen “George” Sawinski

26. Juli 2026

Zusammenfassung

Dieser Kurs ist gedacht für Kinder ab Ende der Grundschule (ca. 5. Klasse), die Begeisterung für Mathematik und Computer mitbringen und den Drang haben, programmieren zu lernen.

Der erste Teil des Kurses beschäftigt sich damit, alle grundlegenden Kenntnisse zu erwerben, um eine Konsolen-basierte Regensimulation in C zu programmieren, was auch mathematische und physikalische Grundlagen beinhaltet.

Der zweite Teil des Kurses wird das Ziel haben, das Spiel Tetris® in C++ umzusetzen, wobei das Hauptaugenmerk dabei auf die Grundlagen Objekt-orientierter Programmierung gelegt wird.

Inhaltsverzeichnis

1 Etwas Mathematik **3**
- 1.1 Das Binärsystem . . . 3
- 1.2 Binäre Operationen — wie Computer rechnen . . . 5

2 C-Grundlagen **7**
- 2.1 Hallo Welt! . . . 7
- 2.2 Zahlen . . . 9
- 2.3 Fibonacci . . . 10
- 2.4 Reihen . . . 12
- 2.5 Lauflicht . . . 13

A Software **14**

B Leitfaden **15**
- B.1 Tastenkombinationen . . . 15
  - B.1.1 Sonderzeichen . . . 15
  - B.1.2 Referenz . . . 16
- B.2 C . . . 17
  - B.2.1 Grundlagen . . . 17
  - B.2.2 Referenz . . . 24

Vorwort

Dieser Kurs ist aus der gemeinsamen Arbeit des Autors mit seinen Söhnen entstanden. Die Idee, einen C/C++-Kurs für Kinder zu schreiben, kam von den Jungs selbst, die unbedingt programmieren lernen wollten um ihre eigenen Spiele zu entwickeln zu können.

Die Mitschrift ist in erster Linie für Kinder gedacht, die den Kurs mit einer Begleitperson zusammen durchlaufen und dient als Leitfaden sowie als Nachschlagewerk für die behandelten Themen.

Der Autor hat die Inhalte erarbeitet, erklärt und in LaTeX festgehalten. Bei der Strukturierung des Materials und der Erstellung einiger LaTeX-Makros wurde er durch KI-gestützte Werkzeuge (**DeepSeek** und **GitHub Copilot**) unterstützt.

DeepSeek lieferte Vorschläge zur Gliederung, zur didaktischen Aufbereitung und zu technischen Umsetzungen in LaTeX. Github Copilot wurde beim Schreiben von LaTeX-Code und bei der Erstellung von C/C++-Codebeispielen eingesetzt. Alle Vorschläge wurden vom Autor geprüft, angepasst und auf die Bedürfnisse seiner Kinder zugeschnitten.

– *Jürgen "George" Sawinski, 26. Juli 2026*

Lizenz und Haftungsausschluss

Alle Inhalte dieses Kurses wurden vom Autor selbst erstellt und sind, soweit nicht anders gekennzeichnet, frei von urheberrechtlich geschützten Materialien Dritter. Die Beispiele und Übungen sind so gestaltet, dass sie die Lernziele unterstützen und den Kindern helfen, die Konzepte zu verstehen und anzuwenden.

Dieses Werk wird unter der **MIT-Lizenz** veröffentlicht. Das bedeutet: Ihr dürft es kopieren, verändern, weitergeben und für eigene Zwecke nutzen — auch kommerziell. Die einzige Bedingung ist, dass der Autor (Jürgen "George" Sawinski) in der Quellenangabe genannt wird.

Haftungsausschluss: Dieser Kurs wurde mit größter Sorgfalt erstellt. Dennoch übernimmt der Autor keine Gewähr für die Richtigkeit, Vollständigkeit oder Aktualität der Inhalte. Die Nutzung der Inhalte erfolgt auf eigene Verantwortung. Der Autor haftet nicht für Schäden, die aus der Nutzung oder der Unfähigkeit zur Nutzung der Inhalte entstehen.

Projekt-Webseite: `https://github.com/jsawinski/twin-c-course`

1 Etwas Mathematik

1.1 Das Binärsystem

Computer rechnen im sogenannten Binärsystem, sie kennen nur 0 und 1. Um größere Zahlen darstellen zu können, werden mehrere dieser Nullen und Einsen, genannt "**Bit**" aneinandergereit. Am gebräuchlichsten dabei ist das "**Byte**" (siehe Abb. 1), das aus 8 Bit besteht.

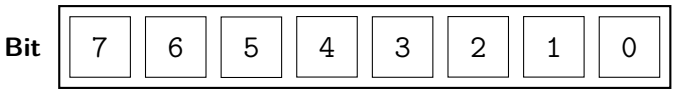


Abbildung 1: Ein Byte mit den Bit-Positionen 0 bis 7.

Jedem Bit wird nun ein Wert zugeordnet. Das erfolgt mathematisch gemäß:

$$2^b$$

wobei b die Nummer des Bits ist. Das ist eine sogenannte "Potenz", eine mathematische Operation die sagt, wie oft eine Zahl mit sich selbst multipliziert wird. In diesem Fall wird die "2" b-mal mit sich selbst multipliziert, zum Beispiel:

$$\begin{aligned} 2^1 &= 2 && = 2 \\ 2^2 &= 2 \cdot 2 && = 4 \\ 2^3 &= 2 \cdot 2 \cdot 2 && = 8 \end{aligned}$$

Interessant ist nun die Frage, was 2^0 ist. Wenn ihr genau hinschaut, dann merkt ihr, dass, um in die nächste Zeile zu kommen, ihr das Ergebnis mit 2 multipliziern müsst. Umgekehrt genauso: um in die vorhergehende Zeile zu kommen, muss man durch 2 teilen. Wenn also 2^1 zwei ist und man, um zu 2^0 zu kommen durch zwei Teilen muss, dann ergibt sich

$$2^0 = 1\,.$$

Übrigens, jede Zahl hoch 0 hat als Ergebnis die 1 (wir kommen später darauf zurück).

Um auf das **Byte** zurückzukommen, heißt dass, dass den **Bits** jeweils die Werte 1 bis 128 zugeordnet sind (siehe Abb. 2).

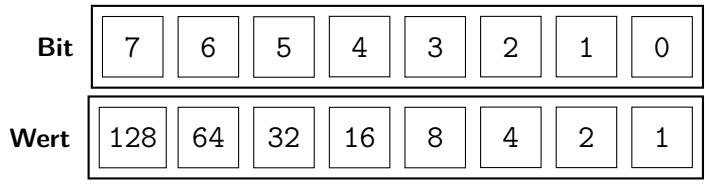


Abbildung 2: Ein Byte mit Bit-Positionen und den zugehörigen Werten (2^b).

Damit können wir nun die Zahlen 0 bis 255 darstellen, z.B.:

$$
\begin{aligned}
00000000 &\to 0\,,\\
00000011 &\to 3\,,\\
01011000 &\to 88\,,\\
11111111 &\to 255\,.
\end{aligned}
$$

Das Addieren funktioniert nun genauso, wie ihr es aus der Schule, in der natürlich das Zehnersystem verwendet wird, gelernt habt. Beispiel:

```
  00100111        39
+ 00000110      +  6
      11          1      (Übertrag)
 ---------       ---
  00101101        45
```

Entsprechend lässt sich auch die Multiplikation durchführen:

```
00001010·00101010 =      10·42
          ..|.|.|.
  00000000.|.|.|.
   00000000|.|.|.
    00001010.|.|.
     00000000|.|.
      00001010.|.
       00000000|.
        00001010.
         00000000
          1 1              (Übertrag)
 ---------------
       110100100          420
```

Wenn ihr genau hinseht, dann bemerkt ihr, dass wir zwei Bytes multipliziert haben, aber das Ergebnis benötigt ein Bit mehr. Das führt zu dem Problem, dass, wenn ihr auf einem Computer diese Multiplikation durchführt, das höchste Bit (die letzte "1" links) abgeschnitten wird und das Ergebnis, dass der Computer euch liefert, 164 ist. Das gleiche passiert bei einem Byte, wenn ihr zu 255 die 1 dazuzählt, das Ergebnis ist 0 (in der Fachsprache als **Überlauf** bezeichnet).

Computer können auch negative Zahlen darstellen. Dazu nützt man aus, dass, wenn $255 + 1$ zum Ergebnis 0 hat, dann hat $0 - 1$ als Ergebnis -1, binär also 11111111. Dies nennt man das "Zweierkomplement" (den Begriff müsst ihr euch natürlich nicht merken). Wichtig ist: dass höchste Bit zeigt, dass es sich um eine negative Zahl handelt. Damit können dann mit einem Byte die Zahlen -128 bis 127 dargestellt werden:

```
10000000        -128
10000001        -127
...
11111111        -  1
00000000           0
00000001           1
...
01111111         127
```

Wie wird nun aus einer positiven Zahl eine Negative? Das passiert, indem man alle Bits invertiert (das heißt, aus 0 wird 1 und aus 1 eine 0) und 1 addiert:

```
(-127)   10000001  ->  01111110
                      +00000001
                       --------
                       01111111      (127)
```

Das funktioniert auch umgekehrt, also aus einer negativen Zahl eine positive zu machen:

```
(127)    01111111  ->  10000000
                      +00000001
                       --------
                       10000001      (-127)
```

1.2 Binäre Operationen — wie Computer rechnen

Bevor wir in das Thema einsteigen, wie Computer rechnen, sollten wir kurz den Begriff "**Operation**" klären.

Eine **Operation** ist eine *Ausführungshandlung*, also ein Vorgang, bei dem aus einem oder mehreren Werten ein neuer Wert berechnet wird. Die **Zeichen**, die diese Operationen beschreiben (wie `+`, `-`, `&`, `|`), nennt man **Operatoren**. Die Werte, mit denen eine Operation arbeitet, heißen **Operanden**. Zum Beispiel sind die Grundrechenarten, die ihr in der Schule gelernt habt, also Addition, Subtraktion, Multiplikation und Division (Plus, Minus, Mal, Geteilt), sogenannte **arithmetische Operationen**.

Obwohl Computer auch diese arithmetische Operationen durchführen können, arbeiten die Schaltungen im Prozessor, dem Herzstück eines Computers, mit sogenannten binären Operationen. Auch schon die ersten Computer, die in den 1940er Jahren gebaut wurden, arbeiteten mit diesen *binären Operationen*[1] (siehe z.B. (1)).

Die grundlegenden binären Operationen sind:

`AND`
: Die `AND`-Operation (binäres UND) verknüpft zwei binäre Zahlen bitweise. Das Ergebnis ist 1, wenn beide Bits 1 sind, sonst ist das Ergebnis 0. Zum Beispiel:

```
  1101
& 1011
--------
  1001
```

`OR`
: Die `OR`-Operation (binäres ODER) verknüpft zwei binäre Zahlen bitweise. Das Ergebnis ist 1, wenn mindestens eines der Bits 1 ist, sonst ist das Ergebnis 0. Zum Beispiel:

[1] Der Autor möchte an dieser Stelle darauf hinweisen, dass diese auch als "logische Operationen" bezeichnet werden, die Begriffe "binäre Operation" und "logische Operation" jedoch nicht immer gleich verwendet werden. Für die Programmiersprache C und C++ gibt es aber eine klare Unterscheidung, auf die wir erst später eingehen werden.

```
   1101
 | 1011
--------
   1111
```

`XOR`

Die `XOR`-Operation (exklusives ODER) verknüpft zwei binäre Zahlen bitweise. Das Ergebnis ist 1, wenn genau eines der Bits 1 ist, sonst ist das Ergebnis 0. Zum Beispiel:

```
   1101
 ^ 1011
--------
   0110
```

`NOT`

Die `NOT`-Operation (binäres NICHT) invertiert die Bits einer binären Zahl. Das Ergebnis ist 1, wenn das Bit 0 ist, und 0, wenn das Bit 1 ist. Zum Beispiel:

```
 ~ 1101
--------
   0010
```

2 C-Grundlagen

2.1 Hallo Welt!

In alter Tradition ist das erste Programm, das man zu sehen bekommt, wenn man eine neue Programmiersprache lernt, gewöhnlich das "Hallo Welt!" Programm (2). Da wir uns zunächst auf die Programmiersprache "C" beschränken, ist das:

```c
/* Das ``Hallo Welt''-Programm. */
#include <stdio.h>

int main() {
  printf("Hallo Welt!\n");
  return 0;
}
```

Erklärung:

`/* Das ``Hallo Welt''-Programm. */`
: Das ist ein **Kommentar**. Der Computer ignoriert ihn völlig. Er ist nur für uns Menschen da, um den Code zu erklären.

  In C gibt es zwei Arten von Kommentaren:

  - `/* ... */` — für längere Kommentare über mehrere Zeilen.
  - `// ...` — für kurze Kommentare in einer Zeile.

`#include <stdio.h>`
: Das ist eine **Header-Datei** (auf Deutsch auch *Kopfdatei*).

  `stdio.h` steht für *Standard Input Output*. Sie enthält Informationen über Funktionen wie `printf`, die wir später benutzen wollen.

  Merke: Wenn wir eine Funktion benutzen wollen, die wir nicht selbst geschrieben haben, müssen wir die dazugehörige Header-Datei einbinden.

`int main() {`
: Das ist die **Hauptfunktion** (englisch: *main function*).

  Jedes C-Programm startet hier. Der Computer führt den Code in den geschweiften Klammern `{ ... }` aus.

  `int` bedeutet, dass die Funktion eine ganze Zahl (Integer) zurückgibt — das ist der Rückgabewert, der weiter unten kommt.

`printf("Hallo Welt!\n");`
: Das ist die **Ausgabe**.

  `printf` steht für *print formatted* und schreibt Text auf den Bildschirm (genauer: in die Konsole).

  Das `\n` ist ein **Zeilenumbruch**. Es sorgt dafür, dass der Cursor nach der Ausgabe in die nächste Zeile springt.

  Wichtig: Jede Anweisung in C endet mit einem Semikolon `;`.

`return 0;`

Das ist die **Rückgabe**.

Sie sagt dem Betriebssystem: *"Alles ist gut gelaufen."*

`0` bedeutet in der Regel *"kein Fehler"*. Wenn ein Programm mit `return 0;` endet, weiß der Computer, dass das Programm erfolgreich war.

`}`

Die schließende geschweifte Klammer beendet die `main`-Funktion. Alles zwischen `{` und `}` gehört zur Funktion.

Zusammenfassung:

- Ein C-Programm beginnt immer in der `main()`-Funktion.
- `printf()` schreibt Text auf den Bildschirm.
- `\n` macht einen Zeilenumbruch.
- `return 0;` sagt dem Computer: "Alles OK."
- Jede Anweisung endet mit `;`.

2.2 Zahlen

Um einen Computer dazu zu bringen, etwas zu tun, müssen wir ihm sagen, was er tun soll. Dazu benutzen wir **Variablen**. Eine Variable ist ein Platzhalter für einen Wert. Wir können ihr einen Namen geben und ihr einen Wert zuweisen. Beispiel: Wir wollen eine Variable namens `zahl` erstellen und ihr den Wert 42 zuweisen. In C sieht das so aus:

```c
#include <stdio.h>

int main() {
  int zahl = 42;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

Dabei ist `int` der Datentyp der Variable, in diesem Fall eine ganze Zahl (Integer). Der Wert 42 wird der Variable `zahl` zugewiesen. Mit `printf` geben wir den Wert der Variable auf dem Bildschirm aus. Das `%d` ist ein Platzhalter für eine ganze Zahl, die an dieser Stelle ausgegeben wird.

Erklärung:

`int zahl = 42;`

Das ist die **Deklaration** und **Initialisierung** einer Variablen.

`int` ist der Datentyp, in diesem Fall eine ganze Zahl (Integer).
`zahl` ist der Name der Variable.
`42` ist der Wert, der der Variable zugewiesen wird.

Es ist auch möglich, eine Variable zu deklarieren, ohne ihr sofort einen Wert zuzuweisen:

```c
  int zahl; // Deklaration ohne Initialisierung

```

und zu einem späteren Zeitpunkt einen Wert zuzuweisen:

```c
  zahl = 42; // Zuweisung eines Wertes

```

`printf("Die Zahl: %d\n", zahl);`

Das ist die **Ausgabe**.

`printf` steht für *print formatted* und schreibt Text auf den Bildschirm (genauer: in die Konsole).

Das `%d` ist ein Platzhalter für eine ganze Zahl, die an dieser Stelle ausgegeben wird.
Das `\n` ist ein **Zeilenumbruch**. Es sorgt dafür, dass der Cursor nach der Ausgabe in die nächste Zeile springt.

2.3 Fibonacci

Die Fibonacci-Zahlen (3) sind eine Folge von Zahlen, bei der jede Zahl die Summe der beiden vorhergehenden ist. Die Folge beginnt mit den Zahlen 0 und 1. Die ersten paar Fibonacci-Zahlen sind:

$$
\begin{aligned}
F(0) &= 0, \\
F(1) &= 1, \\
F(2) &= 1, \\
F(3) &= 2, \\
F(4) &= 3, \\
F(5) &= 5, \\
F(6) &= 8, \\
F(7) &= 13, \\
F(8) &= 21,
\end{aligned}
$$

und so weiter. Die Fibonacci-Zahlen haben viele interessante Eigenschaften und Anwendungen in der Mathematik, der Natur und der Informatik. Sie tauchen in der Natur auf, zum Beispiel in der Anordnung von Blättern an einem Stängel, in der Verzweigung von Bäumen, in der Anordnung von Blütenblättern und in der Struktur von Tannenzapfen.

Die Fibonacci-Zahlen werden folgendermaßen berechnet: Die ersten beiden Zahlen der Folge sind 0 und 1. Jede weitere Zahl ist die Summe der beiden vorhergehenden Zahlen. Hier ist ein Beispielprogramm, das die ersten 16 Fibonacci-Zahlen berechnet und ausgibt:

```c
#include <stdio.h>

int main() {
  int a = 0;
  int b = 1;

  for(int i = 1; i <= 16; i = i+1) {
    int next = a + b;
    a = b;
    b = next;

    printf("Fibonacci %d: %d\n", i, a);
  }
  return 0;
}
```

Erklärung:

`int a = 0;`
: Wir initialisieren die erste Fibonacci-Zahl mit 0.

`int b = 1;`
: Wir initialisieren die zweite Fibonacci-Zahl mit 1.

`for(int i = 1; i <= 16; i = i+1) { … }`
: Wir starten eine Schleife, die 16 Mal durchläuft, um die ersten 16 Fibonacci-Zahlen zu berechnen. Seht auch mal den Abschnitt B.2.1 in der Referenz, um mehr über Schleifen zu erfahren.

`int next = a + b;`
: Wir berechnen die nächste Fibonacci-Zahl, indem wir die beiden vorhergehenden Zahlen addieren.

`a = b;`
: Wir verschieben die Werte: Die aktuelle zweite Zahl wird zur ersten Zahl.

`b = next;`
: Wir verschieben die Werte: Die berechnete nächste Zahl wird zur zweiten Zahl.

`printf("Fibonacci %d: %d\n", i, a);`
: Wir geben die aktuelle Fibonacci-Zahl auf dem Bildschirm aus. Das `%d` ist ein Platzhalter für die aktuelle Zahl, die an dieser Stelle ausgegeben wird.

2.4 Reihen

Die Zahl π ist eine sehr wichtige Zahl in der Mathematik. Sie lässt sich aus einer unendlichen Reihe berechnen:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots$$

Die Reihe geht also unendlich weiter. In der Praxis kann man aber nur endlich viele Terme berechnen. Je mehr Terme man berechnet, desto genauer wird das Ergebnis. In der folgenden C-Datei wird die Reihe bis zum Term 1/1999999 berechnet (das sind etwa sechs Stellen nach dem Komma). Das Ergebnis wird dann mit 4 multipliziert, um π zu erhalten.

```c
#include <stdio.h>

int main() {
    double pi = 1.0;
    for (int i = 1; i < 1000000; i++) {
        double n = 2*i + 1;      // 3, 5, 7, 9, ...

        if (i & 1)
            pi -= 1.0/n;
        else
            pi += 1.0/n;
    }

    printf("Pi: %f\n", 4*pi);
    return 0;
}
```

2.5 Lauflicht

Im folgenden C-Programm wird ein sogenanntes “Lauflicht” erzeugt. Dabei wird ein Sternchen von links nach rechts über den Bildschirm bewegt. Das Programm läuft dabei unendlich lange, bis es mit `Strg+C` abgebrochen wird.

```c
#define __GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

int main() {
    while(1) {
        for (int i = 0; i < 20; i++) {
            printf("\r");
            for (int j = 0; j < 20; j++) {
                if (i == j)
                    printf("*");
                else
                    printf(" ");
            }
            usleep(100000);
        }
    }

    return 0;
}
```

A Software

Als Programmierumgebung wurde VSCode verwendet. Zusätzlich ist es empfehlenswert, die Erweiterung “C C++ Toolkit” zu installieren. Diese Erweiterung bietet unter anderem Syntaxhervorhebung, Autovervollständigung und Debugging-Funktionen für C und C++. Die Möglichkeiten, die VScode bietet, sind sehr umfangreich. Daher ist es empfehlenswert, wenn ihre jemanden fragt, der sich mit VScode auskennt, um die Umgebung optimal einzurichten.

B Leitfaden

B.1 Tastenkombinationen

Tastaturen haben ein paar besondere Tasten, die es einem erlauben, zum Beispiel Text zu markieren, kopieren und einzufügen. Auch lassen sich dadurch einige spezielle Zeichen, die oft bei der Programmierung verwendet werden, die auf einer deutschen Tastatur keine eigenen Tasten belegen, eingeben. Die Tasten, die ihr dafür benötigt, sind:

Shift Die sogenannte "Umschalttaste".[2]Sie schaltet zwischen Klein- und Großschreibung, sowie zwischen den verschiedenen abgebildeten Symbolen um.

Feststelltaste Im Englischen "Caps lock", sie befindet oberhalb der "Shift"-Taste, sie ist dafür gedacht, dauerhaft zwisch Groß- und Kleinbuchstaben umzuschalten.[3]

Strg Die Steuerungstaste (im Englischen "Control" bzw. abgekürzt "Ctrl").

Alt "Alt" steht für "Alternative". Sie wird häufig auch in Verbindung mit "Shift" und "Strg" verwendet. Allein wird sie z.B. verwendet, um zwischen Fenstern mit "Alt+Tab" ("Tab" siehe weiter unten) hin- und herzuschalten.

AltGr "AltGr" steht für "Alternative Grafik", ist im Prinzip die "rechte Alt-Taste", hat aber auch die Funktion, Zeichen einzugeben, die auf, z.B. einer deutschen Tastatur, sonst nicht zur Verfügung stehen.

Esc Das ist die sogenannte "escape"-Taste, auf Deutsch übersetzt "Flucht". Mit dieser Taste lassen sich viele Vorgänge abbrechen.

Cursor Damit sind die Tasten gemeint, mit den man nach rechts, links, oben unten geht. Im Deutschen werden sie auch "Kursor", "Schreibmarke", "Eingabezeiger" usw. bezeichnet, oder einfach als "Pfeiltasten".

Tab Die "Tabulator"-Taste dient dazu, Text richtig einzurücken.

B.1.1 Sonderzeichen

Die folgenden Sonderzeichen gelten für eine deutsche Tastatur:

Tastenkombination	Zeichen	Bedeutung
AltGr + Q	@	At-Zeichen
AltGr + 7	{	Geschweifte Klammer auf
AltGr + 8	[	Eckige Klammer auf
AltGr + 9	]	Eckige Klammer zu
AltGr + 0	}	Geschweifte Klammer zu

[2]Sucht mal auf `https://youtube.com` nach «Umschalttaste mechanische Schreibmaschine», wenn ihr wissen wollt, woher dieser Begriff kommt.

[3]Ihr solltet im Hinterkopf behalten, dass diese Taste selten benutzt wird und auch umgedeutet werden kann. Der Autor verwendet diese Taste auf einer Englischen Tastatur, um komplexe Zeichen einzugeben. Das wird als "Compose-Taste" bezeichnet, z.B. ergibt bei ihm die Eingabe: CapsLock+"+a das Zeichen "ä", oder CapsLock+s+s das Zeichen "ß".

Tastenkombination	Zeichen	Bedeutung
AltGr + ß	\	Backslash
AltGr + <	|	Pipe
AltGr + 3	#	Hashtag
AltGr + +	~	Tilde

B.1.2 Referenz

Die folgende Liste von Tastenkombination gilt für viele, aber nicht für alle Programme:

Tastenkombination	Bedeutung
Markieren	
Esc	Markierung aufheben
Shift+Cursor	Text markieren (alternativ mit Maus)
Strg+A	Alles auswählen ("select all")
Kopieren und Einfügen	
Strg+C	Markierten Text kopieren
Strg+X	Markierten Text ausschneiden
Strg+V	Text einfügen
Strg+Z	Eingabe rückgängig machen ("undo")
Häufige Funktionen	
Strg+S	Speichern (immer wieder drücken!)
Strg+F	Suchen ("find")[4]
Strg+H	Suchen und ersetzen ("replace")

[4]Ihr werdet vielleicht schon einmal in einem Meme "Ctrl+F" gesehen haben. Das bezieht sich in der Tat auf diese Tastenkombination, die allgemein "suchen" bedeutet

B.2 C

B.2.1 Grundlagen

Datentypen

Ganzzahlen C und C++ bieten verschiedene Datentypen für Ganzzahlen an, darunter `int`, `short`, `long` und `long long`. Jeder dieser Typen hat eine unterschiedliche Größe und einen unterschiedlichen Wertebereich. Die Größe hängt von der Architektur des Computers ab, auf dem das Programm ausgeführt wird. Diese sind alle **vorzeichenbehaftet**, was bedeutet, dass sie sowohl positive als auch negative Werte darstellen können. Es gibt auch die vorzeichenlosen Varianten `unsigned int`, `unsigned short`, `unsigned long` und `unsigned long long`, die nur nicht-negative Werte darstellen können (vergleicht das nochmal mit dem Abschnitt 1).

Die Liste der ganzzahligen Datentypen in C und C++ ist wie folgt:

`int`
: Standard-Ganzzahltyp, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: -2.147.483.648 bis 2.147.483.647 (vorzeichenbehaftet) oder 0 bis 4.294.967.295 (vorzeichenlos).

`short`
: Kürzere Ganzzahl, in der Regel 2 Bytes (16 Bit) groß, Wertebereich: -32.768 bis 32.767 (vorzeichenbehaftet) oder 0 bis 65.535 (vorzeichenlos).

`long`
: Längere Ganzzahl, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: -2.147.483.648 bis 2.147.483.647 (vorzeichenbehaftet) oder 0 bis 4.294.967.295 (vorzeichenlos).

`long long`
: Noch längere Ganzzahl, in der Regel 8 Bytes (64 Bit) groß, Wertebereich: -9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807 (vorzeichenbehaftet) oder 0 bis 18.446.744.073.709.551.615 (vorzeichenlos).

Gleitkommazahlen Gleitkommazahlen, also Zahlen mit Nachkommastellen, werden in C und C++ durch die Datentypen `float`, `double` und `long double` dargestellt. Diese Typen unterscheiden sich in der Genauigkeit und dem Wertebereich. Die Liste der Gleitkomma-Datentypen in C und C++ ist wie folgt:

`float`
: Einfachpräzisions-Gleitkommazahl, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: ca. 1.2E-38 bis 3.4E+38

`double`
: Doppelpräzisions-Gleitkommazahl, in der Regel 8 Bytes (64 Bit) groß, Wertebereich: ca. 2.3E-308 bis 1.7E+308

`long double`
: Erweiterte Präzisions-Gleitkommazahl, in der Regel 10, 12 oder 16 Bytes (80, 96 oder 128 Bit) groß, Wertebereich: abhängig von der Implementierung, aber in der Regel größer als `double`

Hier wird eine eigenartige Darstellung von Gleitkommazahlen verwendet, die als **IEEE 754 Standard** bekannt ist. Diese Darstellung ermöglicht es, eine große Bandbreite von Zahlen darzustellen . Das "E" steht für "Exponent" und zeigt an, dass die Zahl in wissenschaftlicher Notation dargestellt wird. Zum Beispiel bedeutet `1.2E-38` die Zahl $1.2 \cdot 10^{-38}$, also eine sehr kleine Zahl, während `3.4E+38` die Zahl $3.4 \cdot 10^{38}$ ist, also eine sehr große Zahl. Man kann sagen, dass die Zahl nach "E" die Zahl der Stellen angibt, um die das Komma verschoben werden muss, um die Zahl in Standardform zu bringen. Ein negativer Exponent bedeutet, dass das Komma nach links verschoben wird, während ein positiver Exponent bedeutet, dass das Komma nach rechts verschoben wird. Damit wird, z.B. die Zahl `1.23E+3` zu 1230 und die Zahl `4.56E-2` zu 0.0456.

Zeichen und Zeichenketten Zeichen werden in C und C++ durch den Datentyp `char` dargestellt. Eine Zeichenvariable speichert ein einzelnes Zeichen und belegt in der Regel 1 Byte. Zeichenketten sind Arrays von Zeichen, die durch das Nullzeichen `'\0'` terminiert sind. Sie werden durch den Datentyp `char[]` oder `char*` dargestellt.

`void` Der Datentyp `void` wird verwendet, um anzugeben, dass eine Funktion keinen Rückgabewert hat. Er kann auch in Zeigern verwendet werden, um einen Zeiger auf einen unbekannten Datentyp zu deklarieren.

Zeiger

`const`

Strukturen Um komplexere Daten zu speichern, können wir in C und C++ **Strukturen** verwenden. Eine Struktur ist eine Sammlung von Variablen, die unter einem gemeinsamen Namen zusammengefasst sind. Jede Variable innerhalb einer Struktur wird als **Mitglied** bezeichnet. Strukturen ermöglichen es uns, verschiedene Datentypen in einer einzigen Einheit zu kombinieren. Hier ist ein Beispiel für eine Struktur, die einen Punkt im zweidimensionalen Raum darstellt:

```c
struct Point {
    int x; // x-Koordinate
    int y; // y-Koordinate
};
```

Wir können dann eine Variable vom Typ `Point` erstellen und auf ihre Mitglieder zugreifen:

```c
struct Point p1; // Erstellen einer Point-Variable
p1.x = 10; // Setzen der x-Koordinate
p1.y = 20; // Setzen der y-Koordinate
printf("Punkt: (%d, %d)\n", p1.x, p1.y); // Ausgabe
```

Alternativ können wir auch die Struktur direkt bei der Deklaration initialisieren:

```c
struct Point p2 = {30, 40}; // Erstellen mit Initialisierung
printf("Punkt: (%d, %d)\n", p2.x, p2.y); // Ausgabe
```

Anweisungsblock Ein Anweisungsblock (englisch: *block*) ist eine Gruppe von Anweisungen, die zusammengefasst und als eine Einheit behandelt werden. In C und C++ wird ein Anweisungsblock durch geschweifte Klammern `{ ... }` definiert. Ein Anweisungsblock kann überall dort verwendet werden, wo eine einzelne Anweisung erwartet wird, z.B. in Kontrollstrukturen wie `if`, `for` oder `while`. Innerhalb eines Anweisungsblocks können Variablen deklariert werden, die nur innerhalb dieses Blocks sichtbar sind (lokale Variablen). Dies ermöglicht eine bessere Organisation des Codes und verhindert Namenskonflikte.

Blöcke können auch verschachtelt werden, d.h. ein Anweisungsblock kann innerhalb eines anderen Anweisungsblocks enthalten sein. Dies ist besonders nützlich, um den Gültigkeitsbereich von Variablen zu begrenzen und die Lesbarkeit des Codes zu verbessern.

Aufpassen solltet ihr jedoch mit der Benennung von Variablen in verschachtelten Blöcken. Wenn ihr in einem inneren Block eine Variable mit dem gleichen Namen wie in einem äußeren Block deklariert, wird die äußere Variable innerhalb des inneren Blocks "verdeckt" (englisch: *shadowed*). Das bedeutet, dass ihr innerhalb des inneren Blocks nur auf die innere Variable zugreifen könnt, während die äußere Variable vorübergehend nicht zugänglich ist. Dies kann zu Verwirrung führen und sollte vermieden werden, indem ihr eindeutige Namen für Variablen verwendet.Beispiel:

```c
int main() {
    int x = 10; // äußere Variable
    {
        int x = 20; // innere Variable verdeckt die Äußere
        printf("Innere x: %d\n", x); // Ausgabe: 20
    }
    printf("Äußere x: %d\n", x); // Ausgabe: 10
    return 0;
}
```

Es sollte angemerkt werden, dass moderne C- und C++-Compiler in der Regel Warnungen ausgeben, wenn Variablen in verschachtelten Blöcken verdeckt werden. Es ist eine gute Praxis, eindeutige Namen für Variablen zu verwenden, um Verwirrung zu vermeiden.

Kontrollstrukturen Sowohl in C als auch in C++ gibt es verschiedene Kontrollstrukturen, die den Ablauf eines Programms steuern. Dazu gehören bedingte Anweisungen wie `if`, `else if` und `else`, sowie Schleifen im folgenden Abschnitt. Diese Strukturen ermöglichen es, Entscheidungen zu treffen und bestimmte Codeabschnitte wiederholt auszuführen, basierend auf Bedingungen oder Zählvariablen.

`if`

Die `if`-Anweisung wird verwendet, um eine Bedingung zu überprüfen und einen Codeblock auszuführen, wenn die Bedingung wahr ist. Die Syntax lautet:

```c
if (Bedingung) {
    // Code, der ausgeführt wird, wenn die Bedingung wahr
    ist
}
```

`else if`
: Die `else if`-Anweisung wird verwendet, um eine weitere Bedingung zu überprüfen, wenn die vorherige `if`-Bedingung falsch war. Die Syntax lautet:

```
if (Bedingung1) {
    // Code, der ausgeführt wird, wenn Bedingung1 wahr ist
} else if (Bedingung2) {
    // Code, der ausgeführt wird, wenn Bedingung2 wahr ist
}
```

`else`
: Die `else`-Anweisung wird verwendet, um einen Codeblock auszuführen, wenn keine der vorherigen Bedingungen wahr ist. Die Syntax lautet:

```
if (Bedingung1) {
    // Code, der ausgeführt wird, wenn Bedingung1 wahr ist
} else if (Bedingung2) {
    // Code, der ausgeführt wird, wenn Bedingung2 wahr ist
} else {
    // Code, der ausgeführt wird, wenn keine der
    Bedingungen wahr ist
}
```

Eine **Bedingung** ist ein Ausdruck, der entweder `true` (wahr) oder `false` (falsch) ergibt. In C und C++ werden Bedingungen durch Vergleiche oder sogenannte logische Ausdrücke gebildet.

Ein Vergleich bildet sich aus zwei Werten und einem Vergleichsoperator. Wenn wir zum Beispiel feststellen wollen ob zwei Zahlen `a` und `b` gleich sind, dann schreiben wir `a == b`. Dabei wird `==` als Operator bezeichnet (um genau zu sein, als Vergleichsoperator).

Liste aller Vergleichsoperatoren: Mögliche Vergleiche sind:

`==`
: Gleichheit: Überprüft, ob zwei Werte gleich sind.

`!=`
: Ungleichheit: Überprüft, ob zwei Werte ungleich sind.

`<`
: Kleiner als: Überprüft, ob der linke Wert kleiner als der rechte Wert ist.

`>`
: Größer als: Überprüft, ob der linke Wert größer als der rechte Wert ist.

`<=`
: Kleiner oder gleich: Überprüft, ob der linke Wert kleiner oder gleich dem rechten Wert ist.

`>=`
: Größer oder gleich: Überprüft, ob der linke Wert größer oder gleich dem rechten Wert ist.

Logische Operatoren werden verwendet, um mehrere Bedingungen zu kombinieren oder zu negieren. Die wichtigsten logischen Operatoren in C und C++ sind:

`&&`
: Logisches UND: Gibt `true` zurück, wenn beide Bedingungen wahr sind.

`||`
: Logisches ODER: Gibt `true` zurück, wenn mindestens eine der Bedingungen wahr ist.

`!`
: Logisches NICHT: Negiert die Bedingung, d.h. gibt `true` zurück, wenn die Bedingung falsch ist, und `false` zurück, wenn die Bedingung wahr ist.

Schleifen C und C++ bieten drei Arten von Schleifen an: `for`, `while` und `do-while`. Jede Schleife hat ihre eigenen Anwendungsfälle und Syntax.

`for`-Schleife
: Die `for`-Schleife wird verwendet, wenn die Anzahl der Schritte im Voraus bekannt ist. Sie hat die folgende Syntax:

```
for(Initialisierung; Bedingung; Inkrement) {
    // Code, der wiederholt ausgeführt wird
}
```

`while`-Schleife
: Die `while`-Schleife wird verwendet, wenn die Anzahl der Schritte nicht im Voraus bekannt ist und die Schleife so lange ausgeführt werden soll, wie eine bestimmte Bedingung erfüllt ist. Sie hat die folgende Syntax:

```
while(Bedingung) {
    // Code, der wiederholt ausgeführt wird
}
```

`do-while`-Schleife
: Die `do-while`-Schleife ist ähnlich wie die `while`-Schleife, aber der Code innerhalb der Schleife wird mindestens einmal ausgeführt, bevor die Bedingung überprüft wird. Sie hat die folgende Syntax:

```
do {
    // Code, der wiederholt ausgeführt wird
} while(Bedingung);
```

Die `for`-Schleife nimmt hier eine besondere Stellung ein. Am besten erklärt man die `for`-Schleife, indem man sie in eine `while`-Schleife umwandelt. Die `for`-Schleife

```
for(Initialisierung; Bedingung; Inkrement) {
    // Code, der wiederholt ausgeführt wird
}
```

ist äquivalent zu:

```
{
    Initialisierung;
    while(Bedingung) {
        // Code, der wiederholt ausgeführt wird
        Inkrement;
    }
}
```

Die **Initialisierung** wird nur einmal zu Beginn der Schleife ausgeführt. Sie dient gewöhnlich dazu, eine Zählvariable zu deklarieren und zu initialisieren. Die **Bedingung** wird vor jedem Durchlauf überprüft, und das **Inkrement** wird am Ende jedes Durchlaufs ausgeführt (Inkrement heißt wörtlich "Erhöhung"). Wenn wir zum Beispiel 1 bis 10 zählen wollen, dann schreiben wir:

```
for(int i = 1; i <= 10; i++) {
    printf("%d\n", i);
}
```

oder von 100 in 10-er Schritten bis 0 zählen wollen, dann schreiben wir:

```
for(int i = 100; i >= 0; i -= 10) {
    printf("%d\n", i);
}
```

Funktionen Um ein Programm in kleinere, wiederverwendbare Teile zu zerlegen, können wir in C und C++ **Funktionen** verwenden. Eine Funktion ist ein benannter Codeblock, der eine bestimmte Aufgabe ausführt und optional Werte zurückgeben kann. Funktionen ermöglichen es uns, den Code zu organisieren, die Lesbarkeit zu verbessern und die Wiederverwendbarkeit zu erhöhen.

Eine Funktion besteht aus einem Funktionskopf, der den Namen der Funktion, die Parameter und den Rückgabewert definiert, und einem Funktionskörper, der die Anweisungen enthält, die ausgeführt werden, wenn die Funktion aufgerufen wird. Die Syntax für die Definition einer Funktion lautet:

```
Rückgabewert Funktionsname(Parameterliste) {
    // Anweisungen
}
```

Der **Rückgabewert** gibt den Datentyp des Wertes an, den die Funktion zurückgibt (siehe oben B.2.1). Wenn die Funktion keinen Wert zurückgibt, wird der Rückgabewert als `void` angegeben. Die **Parameterliste** enthält die Eingabewerte, die an die Funktion übergeben werden, und kann leer sein, wenn keine Eingaben erforderlich sind. Ein Parameter, häufig auch als Argument bezeichnet, ist eine Variable, die innerhalb der Funktion verwendet wird und deren Wert beim Aufruf der Funktion übergeben wird. Ein Argument setzt sich aus Datentype und Variablendamen zusammen. Die Parameterliste kann mehrere Parameter enthalten, die durch Kommas getrennt sind.

Zum Beispiel definieren wir eine Funktion, die zwei ganze Zahlen addiert und das Ergebnis zurückgibt:

```c
int addiere(int a, int b) {
    return a + b; // Rückgabe der Summe
}
```

Wir können diese Funktion dann in unserem Hauptprogramm aufrufen:

```c
int main() {
    int ergebnis = addiere(5, 3); // Aufruf der Funktion
    printf("Ergebnis: %d\n", ergebnis); // Ausgabe
    return 0;
}
```

B.2.2 Referenz

`printf`

Syntax:

```
#include <stdio.h>
int printf(const char *format, ...);
```

Beschreibung:
`printf()` schreibt formattierten Text in die Standardausgabe (stdout). Der `format`-String enthält:

- Normale Zeichen (werden direkt ausgegeben)
- Format-Specifiers (beginnen mit `%`)

Die wichtigsten Format-Specifiers:

`%d`	Ganzzahl (dezimal)
`%f`	Gleitkommazahl
`%c`	Zeichen (char)
`%s`	Zeichenkette (String)
`%x`	Hexadezimal (klein)
`%X`	Hexadezimal (groß)
`%p`	Zeiger (Pointer)

Beispiel:

```c
#include <stdio.h>

int main() {
    int zahl = 42;
    printf("Die Zahl ist: %d\n", zahl);
    printf("Adresse: %p\n", (void*)&zahl);
    return 0;
}

```

Rückgabewert:
Die Anzahl der ausgegebenen Zeichen, oder `-1` bei Fehler.

Siehe auch:
`scanf()`, `sprintf()`, `fprintf()`

Literatur

[1] Wikipedia. Computer, 2003. URL `https://de.wikipedia.org/wiki/Computer`.

[2] Wikipedia. Hallo-Welt-Programm, 2003. URL `https://de.wikipedia.org/wiki/Hallo-Welt-Programm`.

[3] Wikipedia. Fibonacci-Folge, 2003. URL `https://de.wikipedia.org/wiki/Fibonacci-Folge`.